

Lab03 Kaleo256333

Kaleo Campos

2026-08-31

```
library(readr)
library(dplyr)
```

Questão 1

Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

-Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE; -Para o arquivo de vôos: -Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY; -Leia apenas 1 milhão de linhas por vez; -Remova vôos em que o aeroporto de destino comece com a letra 1; -Remova registros em que existam pelo menos uma coluna faltante; -Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino; -Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
airports <- read_csv(
  "airports.csv",
  col_types = cols_only(
    IATA_CODE = "c",
    CITY      = "c",
    STATE     = "c",
    LATITUDE  = "d",
    LONGITUDE = "d"
  )
)

airlines <- read_csv("airlines.csv")

getStats <- function(input, pos) {
```

```

input %>%
  filter(!startsWith(DESTINATION_AIRPORT, "1"), !is.na(DESTINATION_AIRPORT), !is.na(ARRIVAL_DELAY))
  group_by(DESTINATION_AIRPORT) %>%
  summarise(soma = sum(ARRIVAL_DELAY), n = n(), .groups = "drop")
}

computeStats <- function(stats) {
  stats %>%
    group_by(DESTINATION_AIRPORT) %>%
    summarise(soma = sum(soma), n = sum(n), .groups = "drop") %>%
    mutate(atraso_medio = soma / n) %>%
    select(DESTINATION_AIRPORT, atraso_medio)
}

flights_tipos <- cols_only(
  DESTINATION_AIRPORT = "c",
  ARRIVAL_DELAY       = "d"
)

stats_por_lote <- read_csv_chunked(
  file       = "flights.csv",
  callback   = DataFrameCallback$new(getStats),
  chunk_size = 1e6,
  col_types  = flights_tipos
)

flights <- computeStats(stats_por_lote)
flights

```

```

# A tibble: 322 x 2
  DESTINATION_AIRPORT atraso_medio
  <chr>                <dbl>
1 ABE                  5.80
2 ABI                  4.08
3 ABQ                  5.61
4 ABR                 -3.39
5 ABY                  8.68
6 ACK                  4.91
7 ACT                 10.6
8 ACV                  9.68
9 ACY                 11.8
10 ADK                 -3.19

```

```
# i 312 more rows
```

Questão 2

Selecione a operação apropriada join para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

-Identifique a coluna que é a chave na tabela `flights`; -Identifique a coluna que é a chave na tabela `airports`; -Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`? -Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves; -Armazene a tabela resultante no objeto `flights`.

A chave em `flights` é `DESTINATION_AIRPORT`; a chave em `airports` é `IATA_CODE`. Como os nomes das colunas são diferentes nas duas tabelas, o argumento `by` precisa indicar as chaves explicitamente, na forma `c("DESTINATION_AIRPORT" = "IATA_CODE")`.

```
anti_join(flights, airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
```

```
# A tibble: 0 x 2
```

```
# i 2 variables: DESTINATION_AIRPORT <chr>, atraso_medio <dbl>
```

Como os aeroportos de destino codificados apenas como números (iniciados por “1”) já foram removidos na Questão 1, o `anti_join` acima não retorna nenhuma linha: não há aeroportos listados em `flights` que estejam ausentes em `airports`.

```
flights <- flights %>%  
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
```

```
flights
```

```
# A tibble: 322 x 6
```

	DESTINATION_AIRPORT	atraso_medio	CITY	STATE	LATITUDE	LONGITUDE
	<chr>	<dbl>	<chr>	<chr>	<dbl>	<dbl>
1	ABE	5.80	Allentown	PA	40.7	-75.4
2	ABI	4.08	Abilene	TX	32.4	-99.7
3	ABQ	5.61	Albuquerque	NM	35.0	-107.
4	ABR	-3.39	Aberdeen	SD	45.4	-98.4
5	ABY	8.68	Albany	GA	31.5	-84.2
6	ACK	4.91	Nantucket	MA	41.3	-70.1
7	ACT	10.6	Waco	TX	31.6	-97.2
8	ACV	9.68	Arcata/Eureka	CA	41.0	-124.

```

 9 ACY                11.8 Atlantic City NJ        39.5    -74.6
10 ADK                -3.19 Adak                AK        51.9    -177.
# i 312 more rows

```

Questão 3

Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```

airports %>%
  group_by(STATE) %>%
  summarise(n = n(), .groups = "drop") %>%
  arrange(desc(n)) %>%
  knitr::kable()

```

STATE	n
TX	24
CA	22
AK	19
FL	17
MI	15
NY	14
CO	10
MN	8
MT	8
NC	8
ND	8
PA	8
WI	8
GA	7
IL	7
LA	7
VA	7
ID	6
WY	6
AL	5
HI	5
IA	5
MA	5
MO	5
MS	5

STATE	n
OH	5
OR	5
TN	5
UT	5
AR	4
AZ	4
IN	4
KS	4
KY	4
NM	4
SC	4
WA	4
NE	3
NJ	3
NV	3
OK	3
PR	3
SD	3
ME	2
VI	2
AS	1
CT	1
DE	1
GU	1
MD	1
NH	1
RI	1
VT	1
WV	1

Questão 4

Apresente um mapa representando todos os atrasos observados por aeroporto.

-Carregue o pacote leaflet; -Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico; -Armazene o grafico em b) numa variável chamada this; -Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

```
library(leaflet)
library(htmlwidgets)
```

```

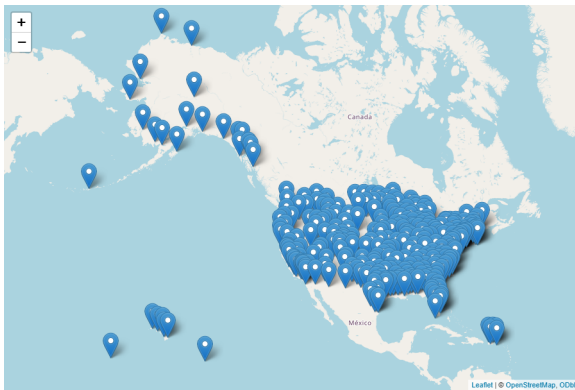
library(webshot2)

chromote::set_chrome_args(c(
  chromote:::default_chrome_args(),
  "--no-sandbox", "--disable-dev-shm-usage", "--disable-gpu"
))

this <- leaflet(data = flights %>% filter(!is.na(LATITUDE), !is.na(LONGITUDE))) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE, lat = ~LATITUDE,
    popup = ~paste0(DESTINATION_AIRPORT, ": ", round(atraso_medio, 1), " min")
  )

saveWidget(this, "mapa.html", selfcontained = TRUE)
invisible(webshot2::webshot("mapa.html", file = "mapa.png", delay = 5, vwidth = 900, vheight = 900))
knitr::include_graphics("mapa.png")

```



```

Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()

```

```
[1] "2026-08-31 23:27:25 -03"
```